

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ
ФЕДЕРАЦИИ
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«КУБАНСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ»
(ФГБОУ ВО «КубГУ»)

Факультет компьютерных технологий и прикладной математики
Кафедра вычислительных технологий

ЛАБОРАТОРНАЯ РАБОТА №6
Дисциплина: «Методы поисковой оптимизации»

Работу выполнили: _____ Долгов М.И. и Костров А.А.

Направление подготовки: 02.03.02 Фундаментальная информатика и
информационные технологии

Преподаватель: _____ Нигодин Е.А.

Краснодар
2026

Тема. Оптимизация алгоритмом иммунной сети.

Цель. Реализовать приложение с графическим интерфейсом, демонстрирующее применение метода оптимизации алгоритмом иммунной сети для поиска точек минимума заданных функций.

Алгоритм иммунной сети. Иммунная сеть представляет собой математическую структуру, имитирующую некоторые функции иммунной системы человека. Многие внешние черты сближают иммунную сеть с такой современной информационной технологией, как нейронные сети. Подобно нейронным сетям, иммунные сети обладают способностью к обучению, прогнозированию и принятию решений в незнакомой ситуации. Как и нейронные сети, иммунные сети не нуждаются в заранее известной модели задачи, а строят эту модель на основе полученной информации. Наконец, как нейронные, так и иммунные сети наиболее эффективны при решении плохо формализованных (слабо структурированных) задач, таких как прогнозирование, классификация и управление.

Популяцию антител (детекторов) обозначим $S_b = (s_i(b), i \in [1 : |S_b|])$, а популяцию антигенов (данных) — $S_g = (s_j(g), j \in [1 : |S_g|])$. Положим, что текущие координаты антитела $s_i(b)$ и антигена $s_j(g)$ определяют, соответственно, векторы

$$X_i(b) = (x_{i,k}, k \in [1 : |X|]), \quad X_j(g) = (x_{j,l}, l \in [1 : |X|]).$$

Аффинность связей антител $s_i(b)$, $s_j(b)$ друг с другом (ВВ-аффинность) определяем как расстояние $\rho(X_i(b), X_j(b)) = \rho_{i,j}(bb)$ между соответствующими векторами. Аналогично определяем ВГ-аффинность связей антитела $s_i(b)$ и антигена $s_j(g)$ в виде расстояния $\rho(X_i(b), X_j(g)) = \rho_{i,j}(bg)$. Полагаем, что меньшим значениям расстояния $\rho(\cdot, \cdot)$ соответствует большая аффинность связей между агентами. Совокупность всех расстояний $\rho_{i,j}(bb)$ обозначаем

$$R_{bb} = \{ \rho_{i,j}(bb), i, j \in [1 : |S_b|] \}.$$

При бинарном кодировании координат агентов в качестве меры расстояния $\rho(\cdot, \cdot)$ обычно используют расстояние Хемминга $\| \cdot \|_H$.

Таким образом, иммунная сеть может быть представлена в виде графа (не обязательно связного), узлы которого соответствуют антителам и антигенам, а дуги — аффинным связям между ними. Формально иммунную сеть определяет набор

$$\langle S_b, S_g, R_{bb}, R_{bg}, S_m, n_b, n_c, b_s, b_b, b_r, b_n \rangle,$$

где кроме введенных выше приняты следующие обозначения:

S^m — подмножество популяции антител S^b , состоящее из клеток памяти иммунной сети;

n_b — число лучших антител, отбираемых для клонирования и мутации из множества антител S^b ;

n_c — число клонов, создаваемых каждым из отобранных антител;

b_s — степень селекции (selection rate), то есть относительное число улучшенных антител, отбираемых из множества клонированных клеток;

b_r — пороговый коэффициент гибели или стимуляции антител в зависимости от значений их r_{bb} -аффинности;

b_b — пороговый коэффициент сжатия иммунной сети;

b_n — коэффициент обновления иммунной сети.

Рассмотрим алгоритм иммунной сети, одна итерация которого имеет следующий вид:

1) Из текущей популяции антител S^b выбираем n_b антител с максимальной BG-аффинностью.

2) Для каждого антитела из числа отобранных n_b антител создаём n_c клонов и помещаем их во множество S^m .

3) Выполняем мутацию каждого антитела из множества S^m по формуле:

$$X_i^m(t+1) = X_i^m(t) + \alpha * U_{-}\{|X_i|\} (-0.5; 0.5), \quad i \in [1 : |S^m|],$$

где α — коэффициент мутации.

4) Определяем BG-аффинность всех антител модифицированного множества S^m и осуществляем его сжатие путем исключения всех антител, кроме n_d лучших.

5) Объединяем популяцию S^b с полученной указанным образом популяцией S^m и выполняем сжатие этой объединенной популяции с сохранением наилучших решений.

Используем данный алгоритм для решения задачи безусловной минимизации функций Розенброка, Химмельбау и Растригина.

1. Программа представляет собой desktop-приложение, реализованное на языке Python с использованием фреймворка PySide6 и библиотеки rpyqtgraph для визуализации. Приложение предназначено для демонстрации алгоритмов оптимизации функций в 3D-пространстве (рисунок 1).

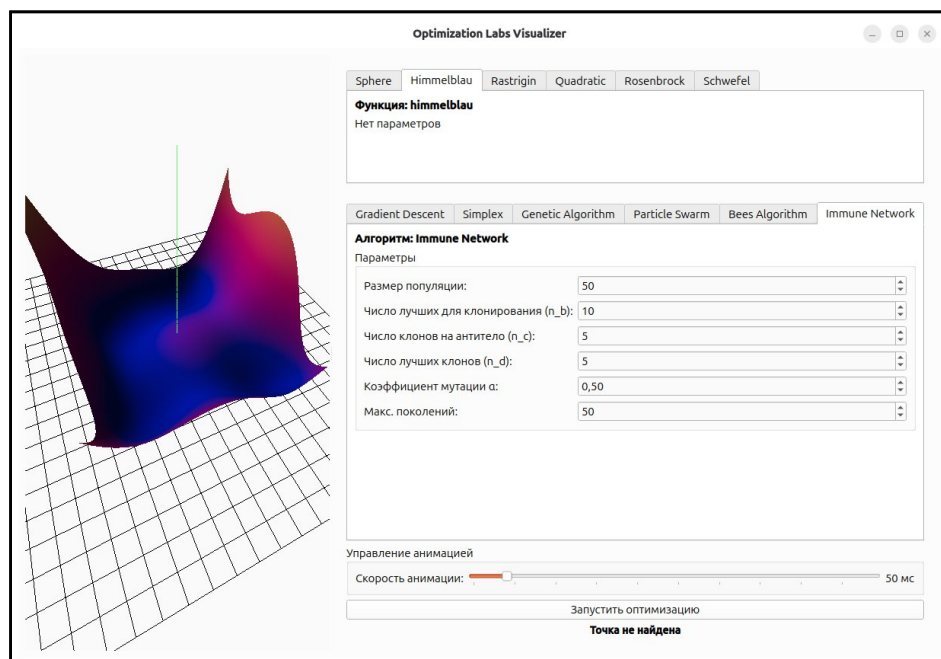


Рисунок 1 — Главное окно приложения

2. В списке функций доступны функции Розенброка, Химмельбау и Растригина (рисунок 2).

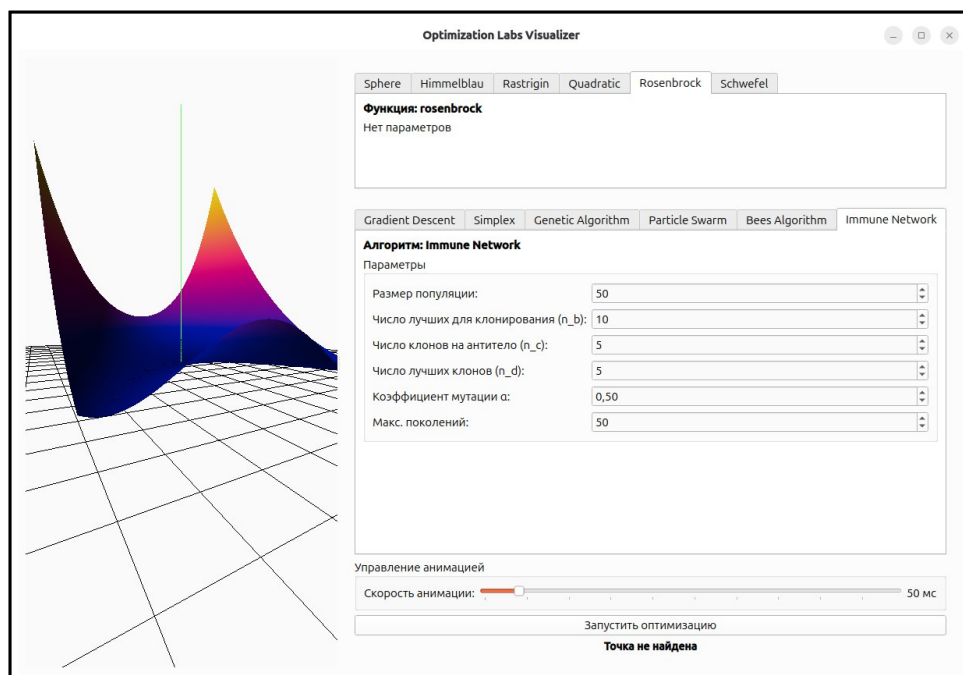


Рисунок 2 — Доступные функции

3. Среди алгоритмов оптимизации доступен алгоритм иммунной сети (рисунок 3).

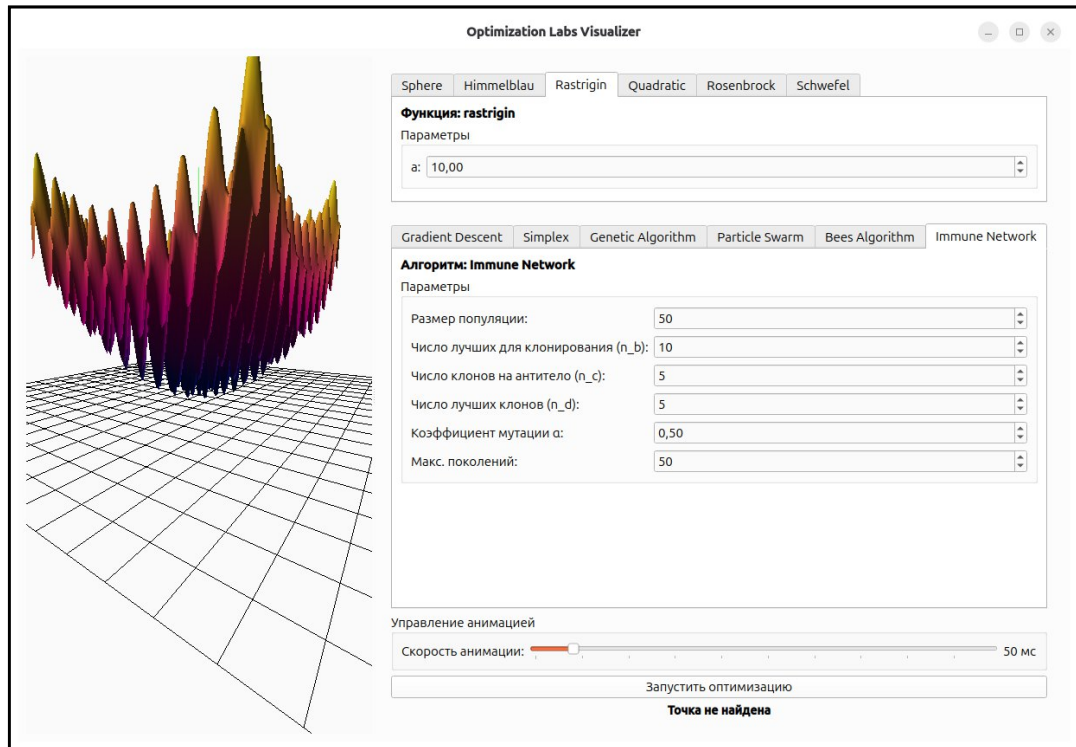


Рисунок 3 — Алгоритм оптимизации

4. Запустим алгоритм поиска минимума функции Розенброка из случайной точки (рисунок 4).

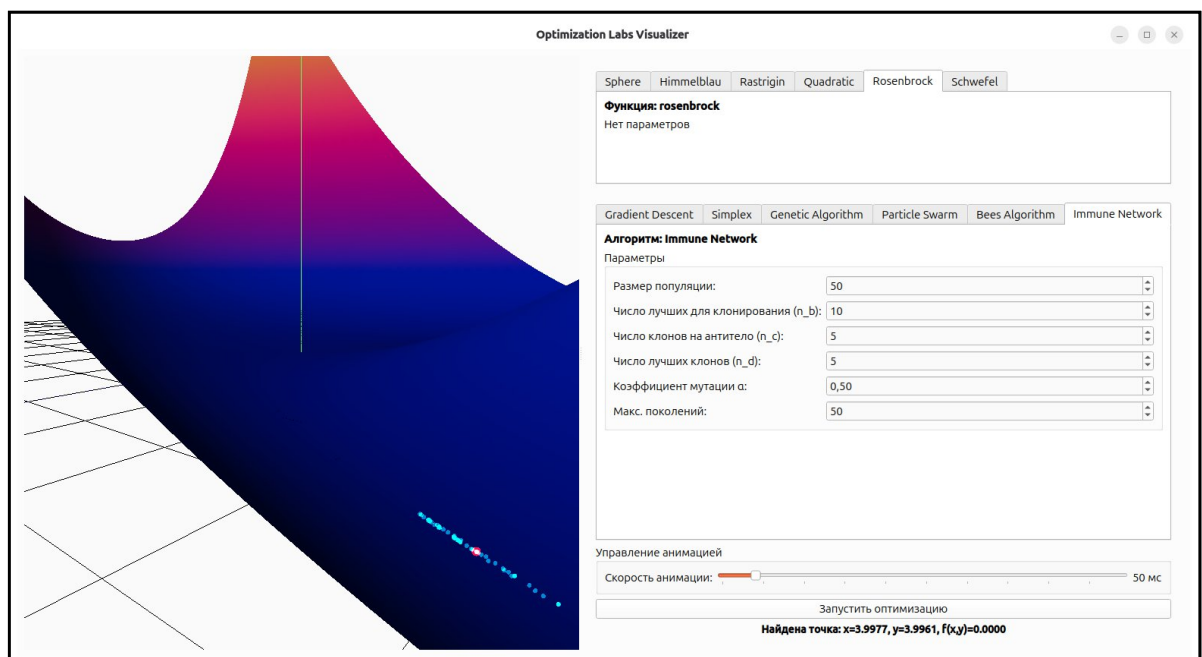


Рисунок 4 — Запуск алгоритма оптимизации

5. Запустим алгоритм поиска минимума функции Растригина из случайной точки (рисунок 5).

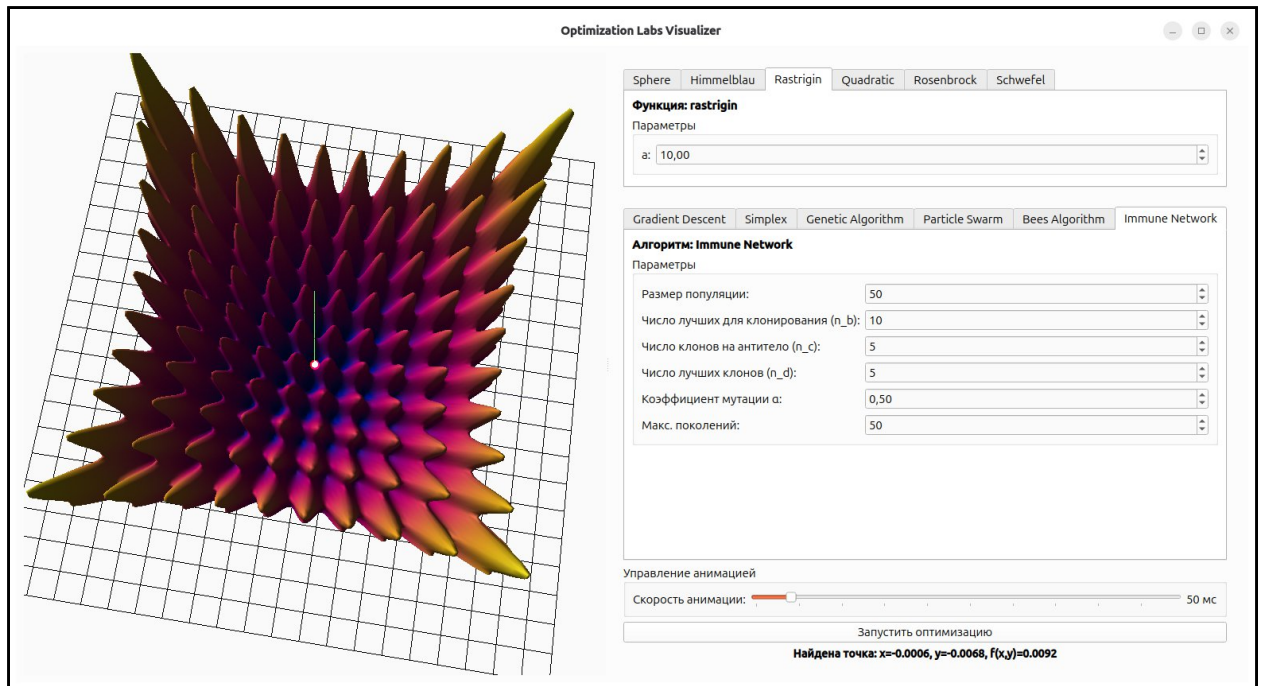


Рисунок 5 — Запуск алгоритма

6. Запустим алгоритм оптимизации с изменёнными параметрами (рисунок 6).

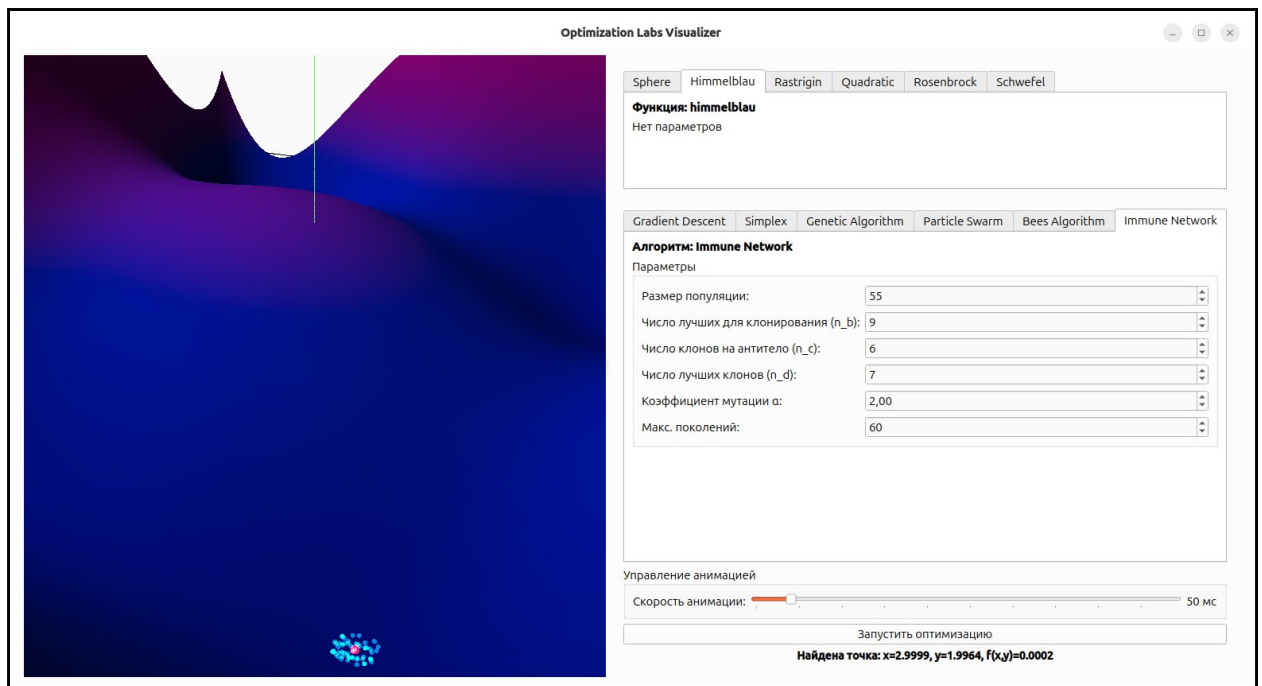


Рисунок 6 — Оптимизация с изменёнными параметрами

Вывод. В ходе данной лабораторной работы было реализовано приложение с графическим интерфейсом, демонстрирующее применение метода оптимизации алгоритмом иммунной сети для поиска минимума заданной функции.

Листинг программы.

```
import numpy as np

def immune_network(
    func,
    x_range: tuple,
    y_range: tuple,
    pop_size: int = 50,
    n_b: int = 10,
    n_c: int = 5,
    n_d: int = 5,
    alpha: float = 0.5,
    generations: int = 50,
    clear_view=None,
    **func_params
):
    """Алгоритм искусственной иммунной сети"""
    # Инициализация популяции
    population = [
        (
            np.random.uniform(x_range[0], x_range[1]),
            np.random.uniform(y_range[0], y_range[1])
        )
        for _ in range(pop_size)
    ]

    # Начальный шаг
    yield sorted(population, key=lambda p: func(*p, **func_params), reverse=True)

    for _ in range(generations):
        if clear_view:
            clear_view()

        # 1) Выбираем n_b лучших (максимальная BG-аффинность = минимальное f)
        fitnesses = [func(*p, **func_params) for p in population]
        sorted_idx = np.argsort(fitnesses)
        best_antibodies = [population[i] for i in sorted_idx[:n_b]]

        # 2) Клонирование
        clones = []
        for ab in best_antibodies:
            clones.extend([ab] * n_c)

        # 3) Мутация каждого клона
        mutated_clones = []
        for x, y in clones:
            dx = alpha * np.random.uniform(-0.5, 0.5)
            dy = alpha * np.random.uniform(-0.5, 0.5)
            nx = max(x_range[0], min(x + dx, x_range[1]))
            ny = max(y_range[0], min(y + dy, y_range[1]))
            mutated_clones.append((nx, ny))

        # 4) Отбор n_d лучших клонов
        clone_f = [func(*p, **func_params) for p in mutated_clones]
        best_clone_idx = np.argsort(clone_f)[:n_d]
        best_clones = [mutated_clones[i] for i in best_clone_idx]

        # 5) Объединение + сжатие (сохраняем лучшие pop_size решений)
        combined = population + best_clones
        combined_f = [func(*p, **func_params) for p in combined]
        best_combined_idx = np.argsort(combined_f)[:pop_size]
        population = [combined[i] for i in best_combined_idx]

    # Возвращаем популяцию (последняя точка — лучшая)
    yield sorted(population, key=lambda p: func(*p, **func_params), reverse=True)
```